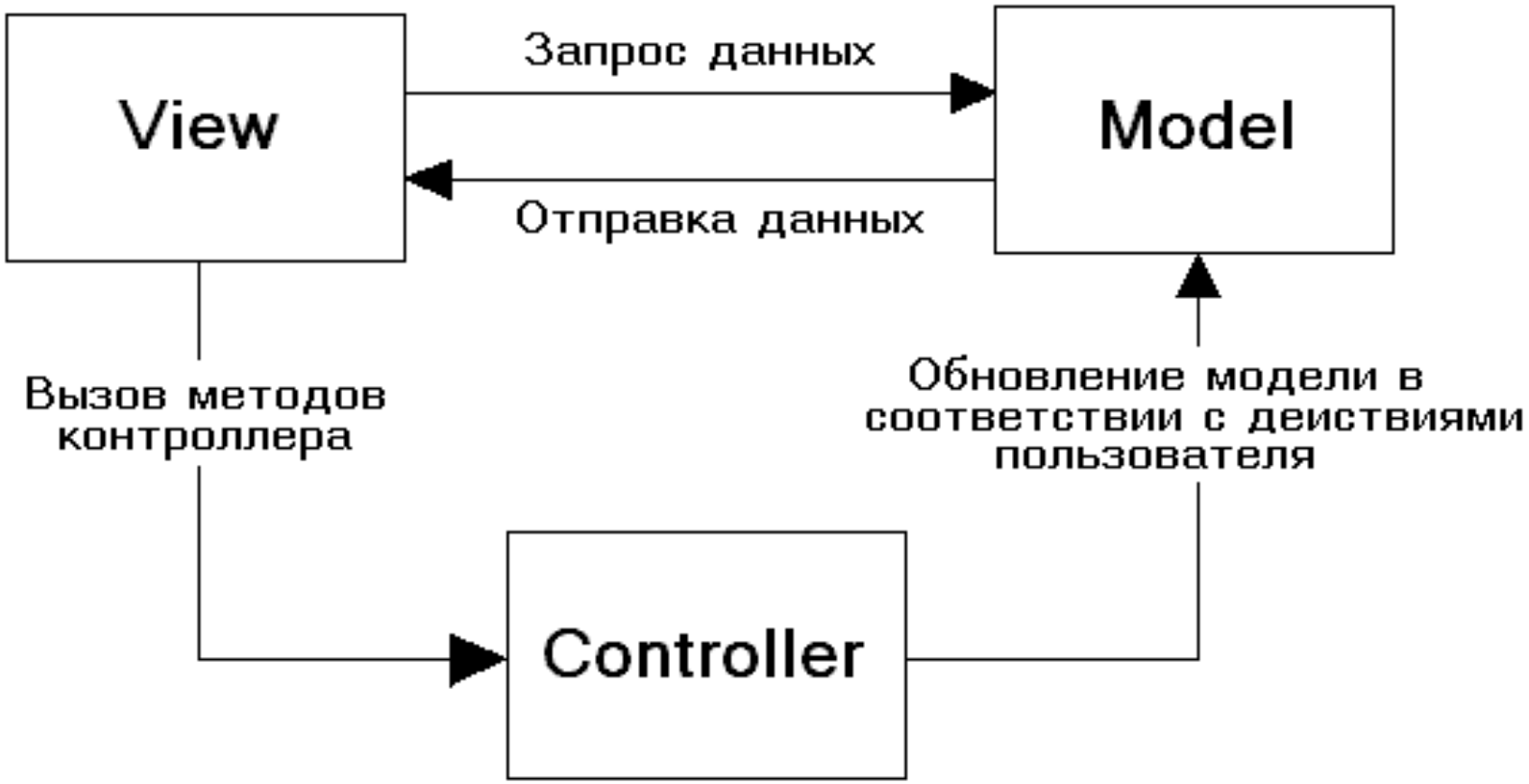
Демидовский университет

Модель-вид-контроллер

Автор: доцент, кандидат технических наук
Ксения Владимировна Лагутина

Модель-вид-контроллер



Демидовский университет

Модель

- Реализует совокупность правил, принципов, зависимостей поведения объектов предметной области (бизнес-логику приложения)
- Не зависит ни от вида, ни от контроллера
- Модель может быть менеджером базы данных, набором объектов или просто логикой приложения
- Модель может быть просто слоем данных (база данных, XML-файл)

Классы модели

- Хранят данные в виде полей
- Имеют методы для получения значений данных в нужном формате
- Имеют методы для изменения данных
- Имеют поля и методы для сообщения об изменении данных (активная модель)

Демидовский университет

Вид (представление)

- Отображает данные полученные от модели
- Не может напрямую влиять на модель
- Часто может быть окном с графическими элементами
- К одной модели можно присоединить несколько видов, при этом не затрагивая реализацию модели
- В некоторых случаях представление может иметь код, который реализует некоторую бизнес-логику

Демидовский университет

Класс вида

- Реализует окно или виджет с графическими потомками
- Получает информацию о данных, хранимых в модели, через ссылку на объект модели
  - Ссылка является полем в классе вида
- Имеет методы для изменения графических элементов при изменении данных в модели

Демидовский университет

Контроллер

- Управляет запросами пользователя
- Вызывает и координирует действия необходимых ресурсов и объектов, нужных для выполнения действий, задаваемых пользователем
  - Обычно контроллер вызывает соответствующую модель и выбирает подходящий вид
- Не затрагивая реализацию видов и модели, можно изменить реакцию на действия пользователя, используя другой контроллер

Демидовский университет

Класс контроллера

- Создаёт и работает с объектами классов модели и вида
- Содержит методы, описывающие обработку действий пользователя
- Инициирует изменение модели, в соответствии с действиями пользователя
- Может сочетать функции контроллера и вида

Демидовский университет

Основные модификации MVC

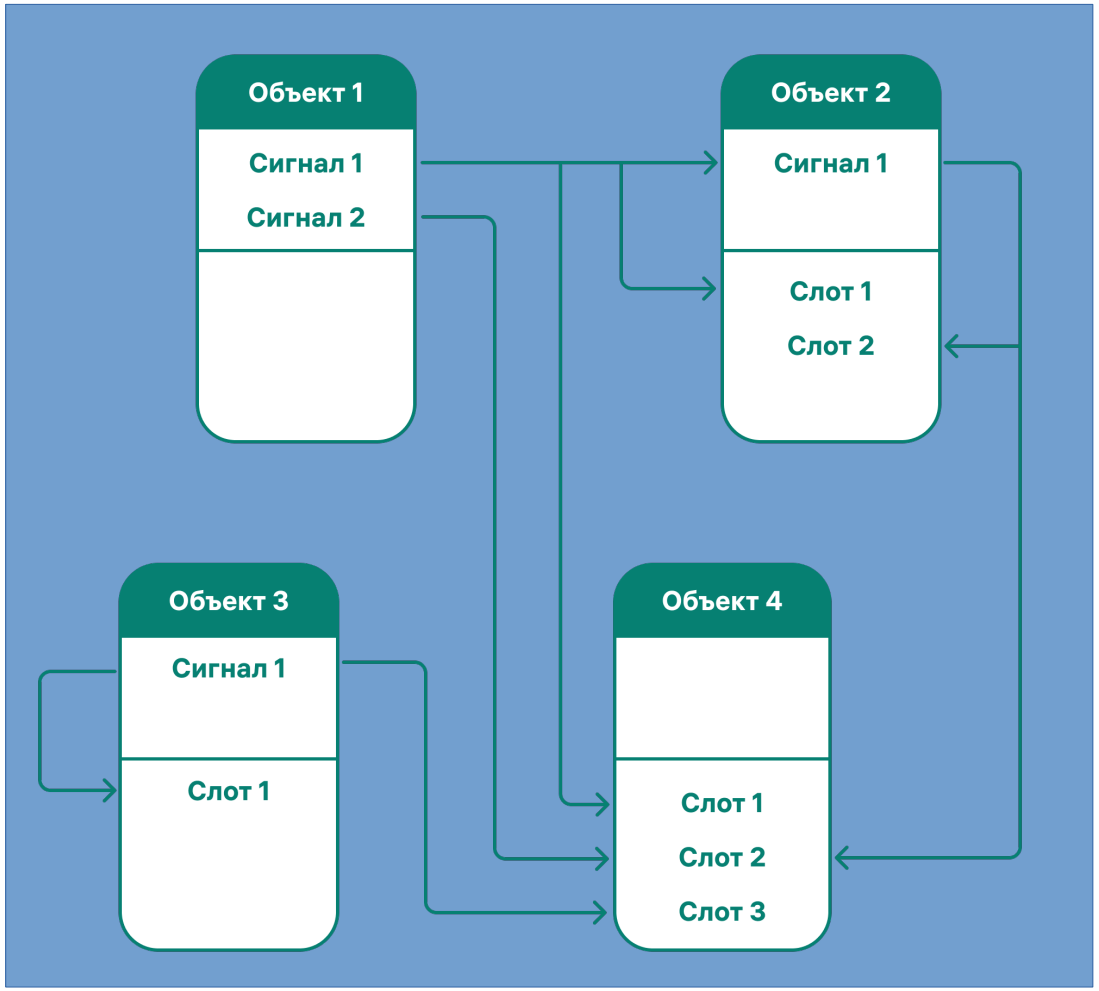
- Пассивная модель
  - не имеет никаких способов воздействовать на вид или контроллер
  - изменения модели отслеживаются контроллером и он же отвечает за перерисовку вида
- Активная модель
  - оповещает вид о том, что в ней произошли изменения

Архитектура модель-вид

- Если классы вида и контроллера объединены, результатом является архитектура модель/вид
- Она по-прежнему отделяет способ хранения данных от способа их представления пользователю, но обеспечивает более простую структуру, основанную на тех же принципах
- Такое разделение позволяет отображать одни и те же данные в разных видах и реализовывать новые типы представлений без изменения базовых структур данных

Демидовский университет

Механизм сигналов и слотов



Демидовский университет

Механизм сигналов и слотов

- Сигналы:
  - Методы, используемые для пересылки сообщений
  - Не должны возвращать значение
  - Сигнал необязательно соединять со слотом
  - Сигнал можно испустить методом emit()
  - Сигнал также имеет возможность передавать параметры
- Слоты:
  - Методы, которые присоединяются к сигналам
  - Не должны возвращать значение
  - Нельзя использовать параметры по умолчанию

Демидовский университет

https://doc.qt.io/qtforpython/tutorials/basictutorial/signals_and_slots.html

Создание сигналов и слотов

```
from PySide6.QtCore import Slot, QObject, Signal, QUrl

class MyClass(QObject):
    signal1 = Signal(float)
    signal2 = Signal(QUrl)
    signal3 = Signal(int, str, int)
    def __init__(self):
        super().__init__()
        self.signal1.connect(self.float_to_int)
        self.signal1.emit(3.5)
    @Slot(float)
    def float_to_int(self, f):
        print(int(f))
example = MyClass()
```

Демидовский университет

Пример

```
from PySide6.QtWidgets import QMainWindow, QApplication
from PySide6.QtCore import Signal, Slot
class Window(QMainWindow):
    closeApp = Signal()
    def __init__(self):
        super().__init__()
        self.closeApp.connect(self.close_by_press)
        self.setGeometry(300, 300, 290, 150)
        self.setWindowTitle('Emit signal')
        self.show()
    def mousePressEvent(self, event):
        self.closeApp.emit()
    @Slot()
    def close_by_press(self):
        print('Я закрываюсь, потому что пользователь нажал на содержимое окна')
        self.close()
if __name__ == '__main__':
    app = QApplication([])
    window = Window()
    window.show()
    app.exec()
```

Демидовский университет

QPushButton

- text(), setText() — текст на кнопке
- setIcon() — добавить иконке кнопку
- Сигналы
  - clicked — кнопка кликнута
  - pressed — кнопка нажата
  - released — кнопка отпущена
- setMenu() — добавить меню
- showMenu() — показать меню (слот)

https://doc.qt.io/qtforpython/PySide6/QtWidgets/QPushButton.html

Демидовский университет

Пример кнопки

```
import sys
from PySide6.QtWidgets import QApplication, QPushButton
from PySide6.QtCore import Slot
@Slot()
def say_hello():
    print("Button clicked, Hello!")
app = QApplication(sys.argv)
button = QPushButton("Click me")
button.clicked.connect(say_hello)
button.show()
app.exec()
```

Демидовский университет

QLineEdit

- text(), setText() — вводимый текст
- setAlignment() — выравнивание текста
- setValidator() — валидация текста
- setPlaceholderText() — подсказка, что вводить, когда текста нет
- setEchoMode() — формат отображения текста (QLineEdit::Normal, QLineEdit::Password)
- Сигналы
  - textChanged — текст изменился
  - textEdited — текст изменён пользователем, а не программно

Демидовский университет

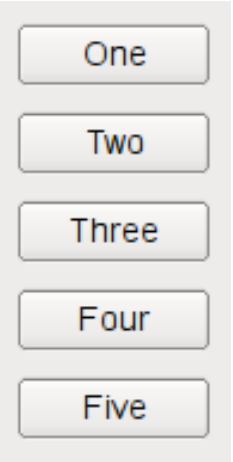
Валидатор

- QIntValidator — проверка целых чисел
  - bottom – нижняя граница
  - top — верхняя граница
- QDoubleValidator — проверка чисел с плавающей точкой
  - bottom – нижняя граница
  - top — верхняя граница
  - decimals — количество знаков после запятой
- QRegularExpressionValidator — валидация по регулярному выражению

Демидовский университет

Позиционирование виджетов

- QVBoxLayout — колонка
- QHBoxLayout — ряд
  - addWidget() — добавить виджет
  - insertItem(index) — добавить виджет на позицию index
  - addLayout() — добавить контейнер в контейнер
- QWidget
  - setLayout() — добавить колонку или ряд в виджет



Демидовский университет

Рекомендации для интерфейсов

- Элементы следует позиционировать относительно, а не абсолютно
- Для вывода текста следует использовать виджеты, которые пользователь не может изменить
  - Не надо использовать QLineEdit для вывода текста
- Следует использовать валидатор для всех обязательных полей ввода
- Кнопкам лучше давать названия с глаголом: «Сохранить данные»

Демидовский университет